

Autonomous Parking in CARLA

Ashik Islam
Electrical Engineering
Cal Poly
San Luis Obispo, USA
akislam@calpoly

William Taing
Electrical Engineering
Cal Poly
San Luis Obispo, USA
wtaing@calpoly.edu

I. MOTIVATION AND OBJECTIVES

Autonomous driving is a quickly developing field of research. Before L5 autonomous vehicles can be safely introduced to the general public, extensive research and technological advances are required. One important capability of a fully autonomous vehicle is the ability to safely park without any human input. The objective of this project is to present one implementation of an autonomous parking system simulation. Specifically, this project focuses on the path planning and control portions of the autonomous parking pipeline. As such, perfect perception and localization are assumed, which also helps limit the scope of the project. This assumption is reasonable in a simulation environment, as growth-truth vehicle pose and predefined parking spot location can be made available.

Video demo: [Click here for video](#)

GitHub: github.com/kiashik/carla_autonomous_parking

II. SIMULATION ENVIRONMENT

This project is developed entirely in the open-source simulator for autonomous driving research called CARLA [1]. CARLA provides urban driving environments, vehicle models, sensor simulation, and tools for validating autonomous driving systems in simulation. The project was developed on Ubuntu 24.04 LTS using Python 3.12, ROS2 Jazzy, and CARLA 0.9.16. Although CARLA 0.9.16 is officially built for Ubuntu 22.04, this project was successfully run on Ubuntu 24.04 without any additional compatibility adjustments. The vehicle used as the test platform was the Toyota Prius model made available in CARLA's catalogue of vehicles.



Figure 1: Parking lot top view

This project also utilizes Nav2 (Navigation2) package, the standard path planning and control stack for ROS2 Jazzy [2]. Nav2 provides planning, control, localization, visualization, and related navigation capabilities for autonomous robotic systems.

In this project, Nav2 was configured to generate a feasible path from the vehicle's current pose to a selected parking spot and to execute the controller needed to track that path.

Town04 was selected as the simulation map because it provides a structured urban environment with a large parking lot suitable for autonomous parking experiments. The selected parking area, shown in Figure 1. It contains clearly defined parking spaces and enough surrounding road space for the vehicle to approach, plan, and execute a parking maneuver. This made it a practical test location for evaluating the full pipeline, including parking spot selection, path planning, and path tracking.

III. METHODOLOGY AND IMPLEMENTATION

The parking system was implemented using a modular ROS2 architecture. CARLA simulation software provides simulated vehicles, a parking lot map, sensor data, and vehicle pose. The data is then processed by the ROS2 nodes for odometry conversion, parking spot occupancy detection, parking spot selection, path planning, path tracking, Ackermann command generation, and vehicle actuation.

The structure of the system is designed to be modular as reflected by the choice to develop within ROS2. The system features five major parts: odometry, path planning, controller, LiDAR-based spot detection and selection, and the occupancy map. Interfacing all these subsystems allows us to consistently complete parking sequences successfully.

A. Odometry

The odometry subsystem is responsible for publishing the vehicle's current position and orientation in the ROS2 environment. CARLA provides access to the simulated vehicle's ground-truth pose, which is converted into ROS2 odometry and transforms message information as it's required for the occupancy map to align with the CARLA map. This step is necessary because both the planner, LiDAR, and controller require a consistent estimate of the vehicle's location within the occupancy map frame.

Since CARLA uses left hand rule and ROS2 uses the right hand rule to define coordinate systems, the odometry node also handles the required axis and yaw transformations such that the vehicle's pose is correctly aligned in RViz2 and throughout the ROS2 navigation stack. All y axis measurements are negated and then shifted accordingly to match the location of the parking lot. All yaw measurements are also negated to match the reflection across the y-axis.

B. Occupancy Map

The occupancy map provides the static representation of the parking lot used for planning and visualization. A simplified map of the CARLA parking lot was generated and loaded into ROS2 as an occupancy grid. Drivable areas are represented separately from occupied or non-drivable regions, allowing the path planner to search only through valid areas of the environment. The occupancy map also provides a common reference frame for the vehicle odometry, parking spot markings, LiDAR detections, and planned path.

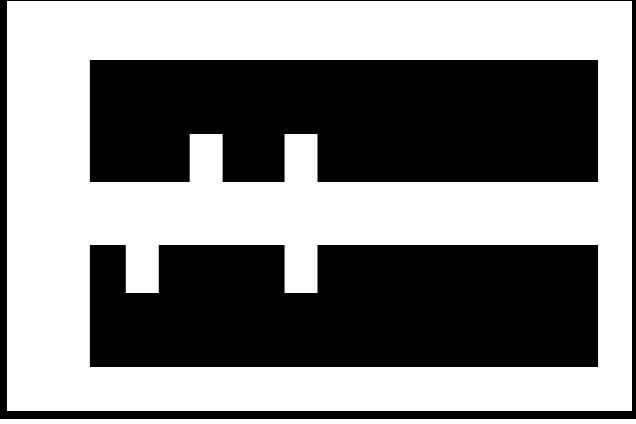


Figure 2: Hardcoded occupancy map of the parking lot in Town 04. Occupied cells are black, and free drivable cells are white.

The occupancy map (see Figure 2) was built such that all parking spots are assigned as occupied to prevent the planner from driving across other free parking spots to reach the goal parking spot. This also prevented the car from planning any paths that would collide with occupied parking spots as well. Only four pre-designated parking spots were classified as free to be selected as goal parking spots. A border of occupied cells was also set to avoid a fence and lampposts located at the edge of the parking lot.

C. Parking Spot Classification

The LiDAR-based spot detection and classification determine which parking spaces are free or occupied. Each parking spot is represented as a predefined goal pose and 3D polygon in the map frame. Simulated LiDAR points are checked against these polygons to determine whether an object is present inside a spot. If enough LiDAR rays fall inside a parking spot volume, the spot is marked as occupied; if the number of points inside is less than the threshold, the spot is marked as free. All other spots too far away or obstructed from the LiDAR are marked as unknown. Once the available spots are identified, the system waits for a start flag to select a valid target parking spot and publishes its goal pose for the planner. The free spot closest to the vehicle is considered the target parking spot. This subsystem allows the vehicle to make a parking decision based on the simulated sensor environment rather than driving to a fixed goal every time.

This subsystem is implemented as ROS node that subscribes to the CARLA LiDAR point cloud, ego odometry, and `/begin_parking` flag. As described in Figure 3, when LiDAR data is received, the point cloud is transformed into the map frame using TF2 and filtered by height to remove ground points

and points above the expected vehicle height. The remaining obstacle points are checked against each parking spot's 3D volume. If enough points fall inside a spot, the spot is marked OCCUPIED. If a visible spot remains below the occupied threshold for several updates, it is marked FREE. Spots outside the LiDAR visible range are marked UNKNOWN.

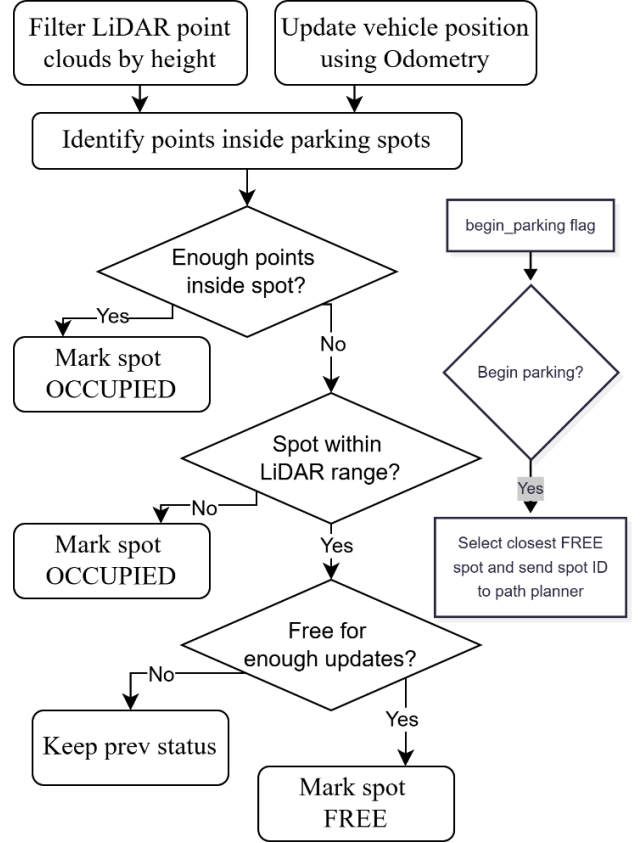


Figure 3: LiDAR-based parking spot classification flowchart.

D. Path Planner

The path planning subsystem generates a feasible path from the vehicle's current pose to the selected parking spot. The planner uses the current odometry, the occupancy map, and the target parking pose as inputs. Since the vehicle is a four-wheeled car, the generated path must respect the vehicle's nonholonomic motion constraints, meaning the car cannot move directly sideways and must follow paths with realistic turning behavior. The planner outputs a path that guides the vehicle from its starting position into the selected parking space while avoiding non-drivable regions in the occupancy map.

The path planner used is the Nav2 Smac Hybrid-A* planner [3]. There are many planner parameters that can be tuned to achieve the desired performance. One of the most important is the Dubins motion model used for search, which allows the planner to create a feasible path based on the car's minimum turning radius and footprint. Dubins was selected over Reeds-Shepp as Dubins disallows reverse motion. Inflation radius also allows us to inflate the area of occupied areas to prevent collisions and increases safety. Many other parameters were tuned for the final planner used.

E. Longitudinal and Lateral Controller

The controller subsystem tracks the planned path and produces the motion commands (throttle and steering) required to drive the vehicle. The controller compares the current vehicle pose with the reference trajectory from the planned path and calculates the steering and speed commands needed to reduce tracking errors. Our implementation uses the Regulated Pure Pursuit (RPP) controller through the Nav2 Library [4]. Regulated Pure Pursuit improves the standard pure pursuit path-tracking method by regulating the linear velocities by the curvature of the path to reduce the overshoot at high speed around corners. It also considers the current velocities to adaptively pick a lookahead distance. The standard pure-pursuit controller is only a lateral controller that regulates the steering angle, but the Regulated Pure Pursuit controller is both a longitudinal and lateral controller. This controller was selected because it is simple, reliable, and allows for active collision detection, making it a suitable for tracking low-speed paths smoothly required for autonomous parking.

In our implementation, the Nav2 controller servers run the `RegulatedPurePursuitController` plugin and publish velocity commands on the `/cmd_vel` topic as `geometry_msgs/Twist` messages. This message holds the desired linear and angular velocities the car should track. However, CARLA's interface expects low-level vehicle commands as a `carla_msgs/CarlaEgoVehicleControl` message. As such, a converter is needed to translate from `geometry_msgs/Twist` to `carla_msgs/CarlaEgoVehicleControl`. This conversion is implemented as a ROS node that does the conversion shown in Figure 4.

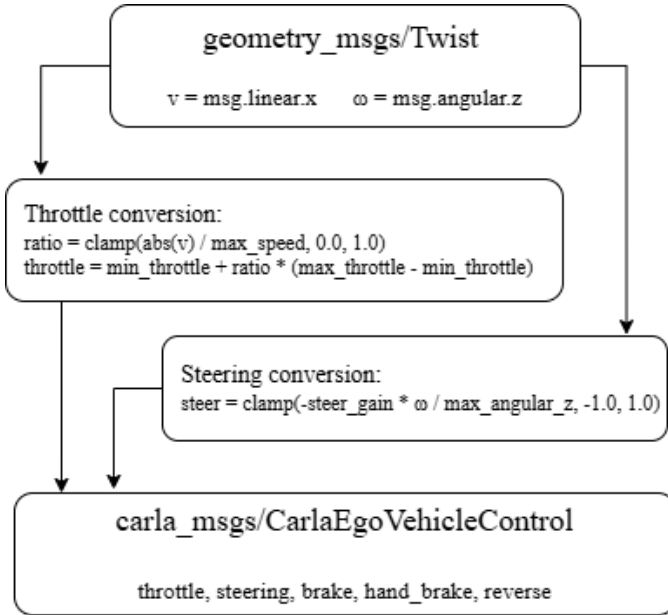


Figure 4: Conversion from `Twist` to `CarlaEgoVehicleControl`. Relevant constants are $\text{max_speed} = 1$ m/s, $\text{max_throttle} = 0.5$, $\text{min_throttle} = 0.1$, $\text{max_angular_z} = 0.4$ rad/s.

More specifically, the conversion node maps the linear velocity to fractional throttle, brake, and reverse logic, and small

velocities inside a deadband result in braking; positive velocity applies forward throttle, and negative velocity enables reverse, but currently, the path planner does not require reversing. The yaw is converted into a normalized steering command using a steering gain and maximum angular velocity limit, with the sign inverted to match CARLA's LHR steering convention. This conversion approach does not work in the general case, but at low speed, it provides a valid conversion that allows the simulated vehicle to track the reference path.

IV. EVALUATION

A. Evaluating the Path Planner

The path planner was able to consistently generate a feasible path from the car's current position to the goal spot. The paths generated also successfully avoided any occupied cells and any collisions with obstacles in the parking lot. The only times that the path planner failed to generate was if the starting position of the car prevented a valid path. For example, if the car is located in an occupied cell or in a position in which the car is stuck and the only way to become unstuck would be to reverse. Although these situations are possible, they are very unlikely when driving normally.

B. Evaluating the Controller

Since the controller output must be converted into throttle and steering commands, the controller's commanded linear and angular velocities cannot be directly compared with the vehicle's actual velocities. This is because the conversion from linear and angular velocity to throttle and steering angle is not generally valid for all vehicle conditions. In addition, the conversion process clips the velocity commands to ensure smooth and slow driving. Therefore, the controller's performance is better evaluated by comparing the reference path with the actual path followed by the vehicle.

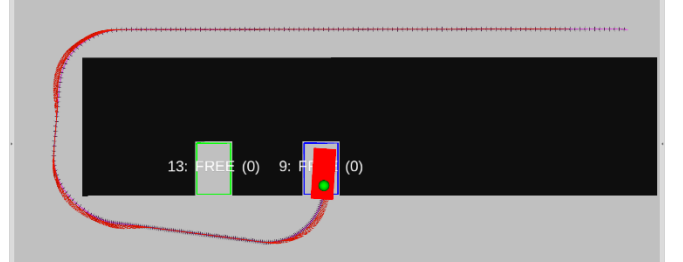


Figure 5: Example of the controller tracking the reference trajectory. The reference trajectory is in purple, and the actual trajectory is represented as red vectors.

Figure 5 shows the overlaid reference trajectory and the actual trajectory followed by the car. The tracking performance is very good, as the reference trajectory is almost entirely hidden behind the actual trajectory. The thicker red lines near the corners may appear slightly offset, but this is because the actual trajectory is represented using vectors rather than a continuous line. This does not indicate poor tracking at the corners.

C. Evaluating Parking Spot Classification

The parking spot classification system was evaluated by observing how accurately the system classified each parking space as either free or occupied from anywhere in the parking lot. Figure 6 shows an example of the parking spot classification status. Spots 13 and 150 are classified as free, as there is no car parked there, and spot 15 is accurately classified as occupied. The number printed next to the label “Free” or “Occupied” is the number of LiDAR rays that hit reflected back from within the 3D volume that defines a parking spot. This visualization confirms that the system can distinguish between empty and occupied parking spaces based on the density of LiDAR returns inside each predefined spot.



Figure 6: Example of parking spot classification using 3D LiDAR.

After classifying the parking spots, the system selects the closest available parking spot as the navigation goal. Figure 7 shows examples of this selection process at different vehicle positions. As the vehicle moves through the parking lot, the selected goal (represented by the green dot) changes based on which free spot is closest to the vehicle. This demonstrates that the system is able to both classify parking availability and update the target parking spot dynamically.

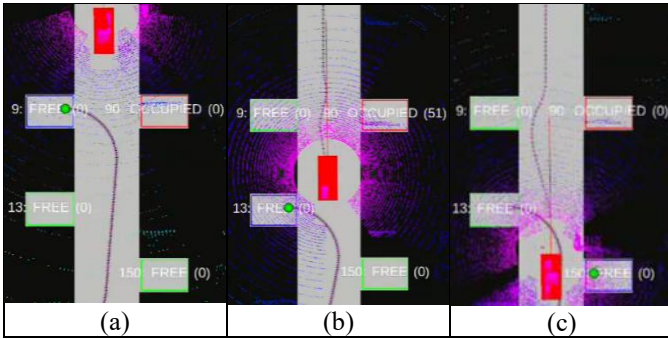


Figure 7: Example of selecting the closest free parking spot. The green dot represents the currently selected goal parking spot. From left to right, the selected parking spots are 9, 13, and 150.

V. CONCLUSION

This project presented a simulation-based autonomous parking system implemented in CARLA using ROS2 and Nav2. The system integrated odometry, a hard-coded occupancy map, LiDAR-based parking spot classification and selection, Hybrid-A* path planning, Regulated Pure Pursuit path tracking, and

CARLA vehicle command conversion. These components allowed the vehicle to identify available parking spots, select the closest free spot, generate a feasible path, and track the path to the goal spot.

The results show that the planner consistently generated feasible paths from valid starting poses, the controller accurately followed the planned trajectory, and the parking spot classification system successfully distinguished between free and occupied spaces. These results demonstrate a complete autonomous parking system.

The system still leaves room for improvement due to its reliance on ground-truth localization, predefined occupancy map, and forward-only planning. Future work could include perception-based spot detection, reverse-capable planning, improved vehicle control, and testing across more varied parking scenarios.

VI. CONTRIBUTION STATEMENT

Both Will and Ashik had equal contributions to the project. Each author is assigned 50 points. Notable contributions from both authors are:

- William:
 - Occupancy map.
 - Path planning.
 - The tuning of the path planner and controller.
- Ashik:
 - Regulated Pure Pursuit Controller.
 - Twist to CarlaEgoVehicleControl: Converting higher-level linear and angular velocity to lower-level fractional throttle and steering command.
 - Parking spot classification,
- Both authors:
 - Setting up the CARLA simulation environment, including configuring Odometry.
 - Setting up Ubuntu and ROS2 workspace
 - Interfacing the subsystems.
 - Video presentation and report.

VII. REFERENCES

- [1] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez and V. Koltun, Proceedings of the 1st Annual Conference on Robot Learning, 2017, pp. 1-16.
- [2] S. Macenski, F. Martin, R. White and J. Ginés Clavero, The Marathon 2: A Navigation System, 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2020.
- [3] S. Macenski, M. Booker and J. F. T. Wallace, "Open-Source, Cost-Aware Kinematically Feasible Planning for Mobile and Surface Robotics," *IEEE Robotics and Automation Practice*, 2026.
- [4] S. Macenski, S. Singh, F. Martin and J. Gines, "Regulated Pure Pursuit for Robot Path Tracking," *Autonomous Robots*, 2023.

